Practical - 11

Practical Title: Student Management CRUD with Search and Filter

Problem Definition: Create a student management module to add, view, edit, delete, search and filter student records. All database operations must use MYSQLi statements and clear success/failure messages.

Theory

1. Are SELECT, INSERT, UPDATE, and DELETE operations working?

Answer: Yes. CRUD operations are used to manage student records in a MySQL database through PHP and MySQLi.

- SELECT: Retrieves and displays student records from the database.
- INSERT: Adds a new student record to the database.
- UPDATE: Modifies the details of an existing student.
- DELETE: Removes a student record from the database.

These operations can be implemented using MySQL queries executed through MySQLi. Success and failure messages should be displayed so users know whether an operation was completed successfully.

Reference Link:
https://www.w3schools.com/php/php_mysql_select.asp
https://www.w3schools.com/php/php_mysql_insert.asp
https://www.w3schools.com/php/php_mysql_update.asp
https://www.w3schools.com/php/php_mysql_delete.asp

2. Is search by name/email/course implemented?

Answer: Yes. A search feature allows the administrator to find student records using a student's name, email address, or course. The entered search term is compared with the corresponding database fields, and only matching records are displayed. For example, searching for WDF can display students enrolled in Web Development Frameworks. Prepared statements should be used where appropriate to handle user input safely and reduce the risk of SQL injection.

Reference Link:
https://www.w3schools.com/php/php_mysql_select.asp
https://www.w3schools.com/php/php_mysql_prepared_statements.asp

3. Are delete operations confirmed before execution?

Answer: Yes. Before deleting a student record, the application should display a confirmation message asking the administrator whether they want to continue. If the administrator confirms, the delete operation is executed; otherwise, the record remains unchanged. This helps prevent accidental deletion of student data. Confirmation can be implemented using JavaScript's confirm() function, while PHP processes the deletion on the server after the request is submitted.

Reference Link:
https://www.w3schools.com/jsref/met_win_confirm.asp
https://www.w3schools.com/php/php_mysql_delete.asp

A. Implementation/ Output/Result

db.php

```
<?php
mysqli_report(MYSQLI_REPORT_ERROR | MYSQLI_REPORT_STRICT);

$host = "localhost";
$username = "root";
$password = "";
$database = "student_management";

try {
    $conn = new mysqli(
        $host,
        $username,
        $password,
        $database
```

```
    );

    $conn->set_charset("utf8mb4");
} catch (mysqli_sql_exception $e) {
    error_log($e->getMessage());
    exit("Database connection failed. Check your configuration.");
}
?>
```

index.php

```
<?php
session_start();
require_once "db.php";

if (empty($_SESSION["csrf_token"])) {
    $_SESSION["csrf_token"] = bin2hex(random_bytes(32));
}

function e($value) {
    return htmlspecialchars(
        (string)$value,
        ENT_QUOTES,
        "UTF-8"
```

```
    );
}

function redirectWithMessage($message) {
    header("Location: index.php?message=" . urlencode($message));
    exit;
}

$message = $_GET["message"] ?? "";
$error = "";
$editStudent = null;

// Handle all form submissions
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $token = $_POST["csrf_token"] ?? "";

    if (!hash_equals($_SESSION["csrf_token"], $token)) {
        $error = "Invalid request. Please refresh and try again.";
    } else {
        $action = $_POST["action"] ?? "";

        try {
```

```
// ADD STUDENT
if ($action === "add") {
    $name = trim($_POST["name"] ?? "");
    $email = trim($_POST["email"] ?? "");
    $course = $_POST["course"] ?? "";
    $year = filter_var(
        $_POST["year"] ?? "",
        FILTER_VALIDATE_INT
    );

    if (
        $name === "" ||
        strlen($name) > 100 ||
        !filter_var($email, FILTER_VALIDATE_EMAIL) ||
        strlen($email) > 100 ||
        !in_array($course, ["WDF", "C++", "DSA"], true) ||
        $year === false ||
        $year < 1 ||
        $year > 4
    ) {
        throw new Exception("Please enter valid student details.");
    }
```

```
    $stmt = $conn->prepare(
        "INSERT INTO students (name, email, course, year)
         VALUES (?, ?, ?, ?)"
    );
    $stmt->bind_param(
        "sssi", $name, $email, $course, $year
    );
    $stmt->execute();

    redirectWithMessage("Student added successfully.");
}

// UPDATE STUDENT
elseif ($action === "update") {
    $id = filter_var(
        $_POST["id"] ?? "",
        FILTER_VALIDATE_INT
    );
    $name = trim($_POST["name"] ?? "");
    $email = trim($_POST["email"] ?? "");
    $course = $_POST["course"] ?? "";
```

```
$year = filter_var(
    $_POST["year"] ?? "",
    FILTER_VALIDATE_INT
);

if (
    $id === false || $id < 1 ||
    $name === "" || strlen($name) > 100 ||
    !filter_var($email, FILTER_VALIDATE_EMAIL) ||
    strlen($email) > 100 ||
    !in_array($course, ["WDF", "C++", "DSA"], true) ||
    $year === false || $year < 1 || $year > 4
) {
    throw new Exception("Please enter valid student details.");
}

$stmt = $conn->prepare(
    "UPDATE students
     SET name = ?, email = ?, course = ?, year = ?
     WHERE id = ?"
);
$stmt->bind_param(
```

```
            "sssii", $name, $email, $course, $year, $id
        );
        $stmt->execute();

        redirectWithMessage(
            $stmt->affected_rows >= 0
                ? "Student updated successfully."
                : "Update could not be completed."
        );
    }

    // DELETE STUDENT
    elseif ($action === "delete") {
        $id = filter_var(
            $_POST["id"] ?? "",
            FILTER_VALIDATE_INT
        );

        if ($id === false || $id < 1) {
            throw new Exception("Invalid student ID.");
        }
```

```
        $stmt = $conn->prepare(
            "DELETE FROM students WHERE id = ?"
        );
        $stmt->bind_param("i", $id);
        $stmt->execute();

        if ($stmt->affected_rows === 1) {
            redirectWithMessage("Student deleted successfully.");
        } else {
            throw new Exception("Student not found.");
        }
    }
} catch (mysqli_sql_exception $ex) {
    if ($ex->getCode() === 1062) {
        $error = "This email address is already registered.";
    } else {
        error_log($ex->getMessage());
        $error = "A database operation failed.";
    }
} catch (Exception $ex) {
    $error = $ex->getMessage();
}
```

```
    }
}

// Load student details into the edit form
if (isset($_GET["edit"])) {
    $id = filter_var($_GET["edit"], FILTER_VALIDATE_INT);

    if ($id !== false && $id > 0) {
        $stmt = $conn->prepare(
            "SELECT id, name, email, course, year
             FROM students WHERE id = ?"
        );
        $stmt->bind_param("i", $id);
        $stmt->execute();
        $editStudent = $stmt->get_result()->fetch_assoc();

        if (!$editStudent) {
            $error = "Student not found.";
        }
    } else {
        $error = "Invalid student ID.";
    }
```

```
}

// SEARCH AND FILTER
$search = trim($_GET["search"] ?? "");
$courseFilter = $_GET["course"] ?? "";

if (!in_array($courseFilter, ["", "WDF", "C++", "DSA"], true)) {
    $courseFilter = "";
}

$where = [];
$params = [];
$types = "";

if ($search !== "") {
    $where[] = "(name LIKE ? OR email LIKE ?)";
    $term = "%" . $search . "%";
    $params[] = $term;
    $params[] = $term;
    $types .= "ss";
}
```

```
if ($courseFilter !== "") {
    $where[] = "course = ?";
    $params[] = $courseFilter;
    $types .= "s";
}

$whereSQL = $where ? " WHERE " . implode(" AND ", $where) : "";

// COUNT matching students for pagination
$countStmt = $conn->prepare(
    "SELECT COUNT(*) AS total FROM students" . $whereSQL
);

if ($types !== "") {
    $countStmt->bind_param($types, ...$params);
}

$countStmt->execute();
$total = (int)$countStmt->get_result()->fetch_assoc()["total"];

$perPage = 5;
$totalPages = max(1, (int)ceil($total / $perPage));
```

```
$page = filter_var($_GET["page"] ?? 1, FILTER_VALIDATE_INT);

if ($page === false || $page < 1) {
    $page = 1;
}

$page = min($page, $totalPages);
$offset = ($page - 1) * $perPage;

// Fetch the current page of students
$sql = "SELECT id, name, email, course, year, created_at
        FROM students" . $whereSQL .
        " ORDER BY id DESC LIMIT ? OFFSET ?";

$listStmt = $conn->prepare($sql);

$listTypes = $types . "ii";
$listParams = array_merge($params, [$perPage, $offset]);
$listStmt->bind_param($listTypes, ...$listParams);
$listStmt->execute();

$students = $listStmt->get_result();
```

```
?>


<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>Student Management CRUD</title>


    <style>
        * { box-sizing: border-box; }


        body {
            font-family: Arial, sans-serif;
            background: #eaf2ff;
            margin: 0;
            padding: 25px;
            color: #222;
        }


        .container {
            max-width: 1100px;
```

```
    margin: auto;
    background: white;
    padding: 25px;
    border-radius: 10px;
}

h1 { color: #0b5ed7; text-align: center; }

.form-grid {
    display: grid;
    grid-template-columns: repeat(2, minmax(0, 1fr));
    gap: 12px;
}

input, select, button, .button {
    width: 100%;
    padding: 10px;
    border-radius: 5px;
    border: 1px solid #bbb;
    font-size: 14px;
}
```

```
button, .button {
    background: #0b5ed7;
    color: white;
    border: none;
    cursor: pointer;
    text-align: center;
    text-decoration: none;
    display: inline-block;
}

.secondary { background: #555; }
.danger { background: #dc3545; }
.success-msg { background: #d1e7dd; padding: 12px; }
.error-msg { background: #f8d7da; padding: 12px; }

.search-row {
    display: flex;
    gap: 10px;
    margin: 20px 0;
}

.search-row input { flex: 2; }
```

```
.search-row select { flex: 1; }

.table-wrap { overflow-x: auto; }

table {
  width: 100%;
  border-collapse: collapse;
  margin-top: 15px;
}

th, td {
  border: 1px solid #ddd;
  padding: 10px;
  text-align: left;
}

th { background: #0b5ed7; color: white; }

.actions {
  display: flex;
  gap: 6px;
  min-width: 150px;
```

```
}

.actions form { flex: 1; }

.actions a, .actions button {
  font-size: 12px;
  padding: 8px 5px;
}

.pagination {
  display: flex;
  flex-wrap: wrap;
  justify-content: center;
  gap: 8px;
  margin-top: 20px;
}

.pagination a {
  padding: 8px 12px;
  background: #e3f2fd;
  color: #0b5ed7;
  text-decoration: none;
```

```
        border-radius: 4px;
      }

      .pagination .current {
        background: #0b5ed7;
        color: white;
      }

      @media (max-width: 600px) {
        body { padding: 10px; }
        .container { padding: 14px; }
        .form-grid { grid-template-columns: 1fr; }
        .search-row { flex-direction: column; }
      }
    </style>
</head>

<body>
<div class="container">
    <h1>Student Management System</h1>
    <p>Add, view, edit, delete, search and filter student records.</p>
```

```
<?php if ($message !== ""): ?>
    <p class="success-msg"><?= e($message) ?></p>
<?php endif; ?>

<?php if ($error !== ""): ?>
    <p class="error-msg"><?= e($error) ?></p>
<?php endif; ?>

<h2><?= $editStudent ? "Edit Student" : "Add New Student" ?></h2>

<form method="post" action="index.php">
    <input type="hidden" name="csrf_token"
           value="<?= e($_SESSION["csrf_token"]) ?>">

    <input type="hidden" name="action"
           value="<?= $editStudent ? "update" : "add" ?>">

    <?php if ($editStudent): ?>
        <input type="hidden" name="id"
               value="<?= e($editStudent["id"]) ?>">
    <?php endif; ?>
```

```
<div class="form-grid">
    <input type="text" name="name" placeholder="Student name"
           maxlength="100" required
           value="<?= e($editStudent["name"] ?? "") ?>">

    <input type="email" name="email" placeholder="Email address"
           maxlength="100" required
           value="<?= e($editStudent["email"] ?? "") ?>">

    <select name="course" required>
       <option value="">Select course</option>
       <?php foreach (["WDF", "C++", "DSA"] as $c): ?>
          <option value="<?= e($c) ?>"
             <?= (($editStudent["course"] ?? "") === $c)
                ? "selected" : "" ?>>
             <?= e($c) ?>
          </option>
       <?php endforeach; ?>
    </select>

    <select name="year" required>
       <option value="">Select year</option>
```

```
            <?php for ($y = 1; $y <= 4; $y++): ?>
                <option value="<?= $y ?>"
                    <?= (string)($editStudent["year"] ?? "") === (string)$y
                        ? "selected" : "" ?>>
                    Year <?= $y ?>
                </option>
            <?php endfor; ?>
        </select>

        <button type="submit">
            <?= $editStudent ? "Update Student" : "Add Student" ?>
        </button>

        <?php if ($editStudent): ?>
            <a class="button secondary" href="index.php">Cancel Edit</a>
        <?php endif; ?>
    </div>
</form>

<h2>Student Records</h2>

<form method="get" action="index.php" class="search-row">
```

```
    <input type="search" name="search"
           placeholder="Search by name or email"
           value="<?= e($search) ?>">

    <select name="course">
      <option value="">All Courses</option>
      <?php foreach (["WDF", "C++", "DSA"] as $c): ?>
        <option value="<?= e($c) ?>"
          <?= $courseFilter === $c ? "selected" : "" ?>>
          <?= e($c) ?>
        </option>
      <?php endforeach; ?>
    </select>

    <button type="submit">Search / Filter</button>
    <a class="button secondary" href="index.php">Reset</a>
</form>

<p>
    Matching records: <strong><?= $total ?></strong>
</p>
```

```
<div class="table-wrap">
    <table>
        <thead>
        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Email</th>
            <th>Course</th>
            <th>Year</th>
            <th>Created</th>
            <th>Actions</th>
        </tr>
        </thead>

        <tbody>
        <?php if ($students->num_rows > 0): ?>
            <?php while ($student = $students->fetch_assoc()): ?>
                <tr>
                    <td><?= e($student["id"]) ?></td>
                    <td><?= e($student["name"]) ?></td>
                    <td><?= e($student["email"]) ?></td>
                    <td><?= e($student["course"]) ?></td>
```

```
                <td><?= e($student["year"]) ?></td>
                <td><?= e($student["created_at"]) ?></td>
                <td>
                    <div class="actions">
                        <a class="button"
                           href="index.php?edit=<?= e($student["id"]) ?>">
                            Edit
                        </a>

                        <form method="post" action="index.php"
                              onsubmit="return confirm('Are you sure you want to
delete this student?');">
                            <input type="hidden" name="csrf_token"
                                   value="<?= e($_SESSION["csrf_token"]) ?>">
                            <input type="hidden" name="action"
                                   value="delete">
                            <input type="hidden" name="id"
                                   value="<?= e($student["id"]) ?>">

                            <button type="submit" class="danger">
                                Delete
                            </button>
                        </form>
```

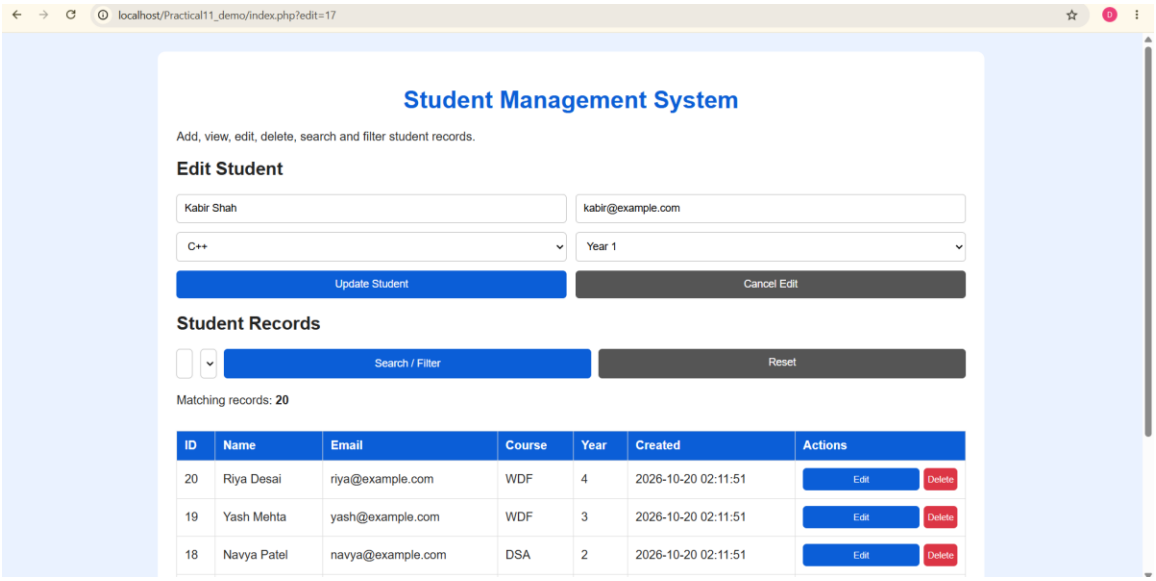
```
                    </div>
                </td>
            </tr>
        <?php endwhile; ?>
    <?php else: ?>
        <tr>
            <td colspan="7">No matching students found.</td>
        </tr>
    <?php endif; ?>
    </tbody>
  </table>
</div>

<?php if ($totalPages > 1): ?>
  <div class="pagination">
    <?php for ($p = 1; $p <= $totalPages; $p++): ?>
      <a class="<?= $p === $page ? "current" : "" ?>"
        href="?<?= e(http_build_query([
          "search" => $search,
          "course" => $courseFilter,
          "page" => $p
        ])) ?>">
```

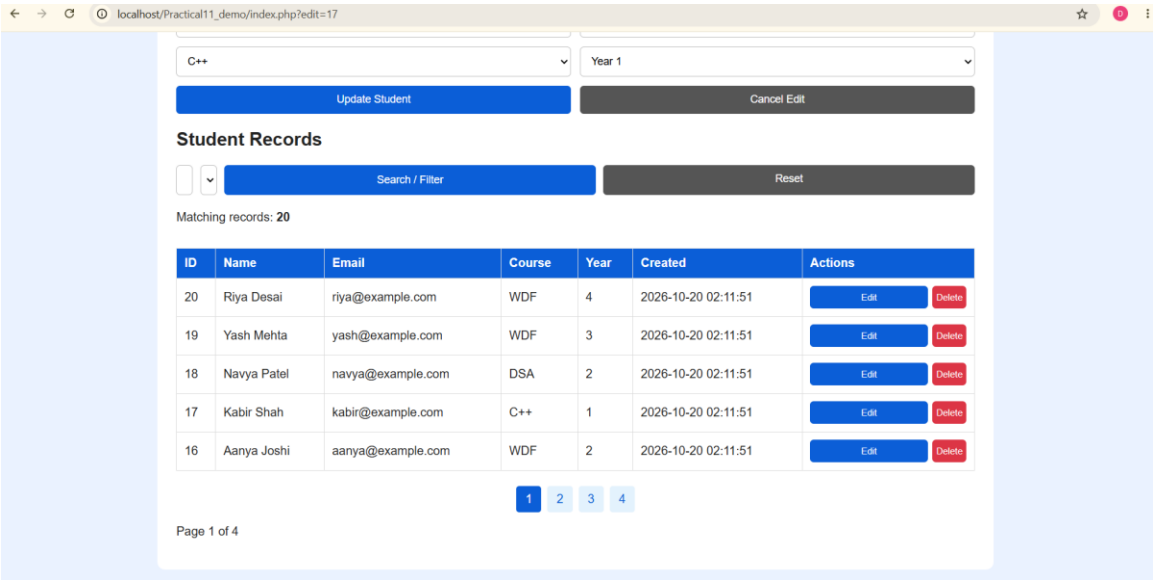
```
                <?= $p ?>
            </a>
        <?php endfor; ?>
    </div>
<?php endif; ?>

<p>Page <?= $page ?> of <?= $totalPages ?></p>
</div>
</body>
</html>
```

B. Evidence of Work (Screenshots)



Description: This screenshot shows the Student Management System developed for Practical 11. The interface provides functionality to add, view, edit, delete, search, and filter student records, fulfilling the main requirements of the CRUD-based student management module. The Edit Student section at the top displays the selected student's existing details, including name, email, course, and year, with options to Update Student or Cancel Edit. This demonstrates the UPDATE operation. Below this, the Student Records section provides search/filter controls and displays the message "Matching records: 20", confirming that the system contains the required minimum of 20 seeded student records. The records are presented in a structured table containing ID, Name, Email, Course, Year, Created date, and Actions. Each record has Edit and Delete buttons, demonstrating the availability of update and delete operations. The displayed records, such as Riya Desai, Yash Mehta, and Navya Patel, also confirm that student data is being successfully retrieved from the MySQL database through the application. Overall, the screenshot demonstrates the working CRUD interface, student record listing, editing functionality, search/filter interface, and record management controls required for Practical 11.



Description: The screenshot shows the Student Management CRUD system developed using PHP and MySQLi. The page displays the student editing form, search and filter options, and a table of student records stored in the database. The edit form allows the user to modify an existing student's details, such as course and academic year, using the Update Student button, while the Cancel Edit button allows the user to cancel the editing process. The Search/Filter section helps users find specific student records based on the available criteria, and the Reset button clears the applied search or filter conditions. The Student Records table displays details such as student ID, name, email, course, year, and creation date, along with Edit and Delete buttons for each record. The screenshot shows 20 matching records, with five records displayed per page and pagination controls to navigate through the list. This practical demonstrates the implementation of CRUD operations using PHP and MySQLi, including retrieving records using SELECT, updating records using UPDATE, and providing options for deletion using DELETE. Overall, the module provides an organized interface for managing student records and demonstrates search, filtering, editing, and pagination functionality as required by Practical 11.

C. GitHub Repository Link:

https://github.com/devanshirshah24-pixel/WDF-Practicals/tree/main/Practical11_Demo

D. Challenges Faced:

- Database connectivity: Connecting PHP with the MySQL database using MySQLi and resolving connection errors was challenging.

- CRUD operations: Understanding and implementing INSERT, SELECT, UPDATE, and DELETE queries correctly required careful handling of student records.

- Search and filter functionality: Displaying the correct student records based on name, email, or course required writing appropriate SQL queries.

- Data validation: Ensuring that student details were entered correctly and required fields were not left empty was important.

- Delete confirmation: Implementing a confirmation message before deleting records helped prevent accidental data loss.

- Success and failure messages: Displaying appropriate messages after database operations helped users understand whether their actions were successful.

- Query security: Using MySQLi prepared statements helped protect the application against SQL injection attacks.

E. Learning Outcomes:

- Connect a PHP application to a MySQL database using MySQLi.

- Perform CRUD operations to insert, view, update, and delete student records.

- Implement search functionality using student names, email addresses, and courses.

- Filter database records according to specific search conditions.

- Use prepared statements to improve database query security.

- Implement confirmation messages before deleting student records.

- Display success and failure messages for database operations.

- Develop a student management page that allows administrators to manage student information efficiently.

Faculty Remarks: (Keep this blank for faculty)